

his need, so we need to consider changes to make it possible without endangering the stability of the system.

Motivation

One of the primary examples provided by users seeking tracking of vulnerabilities across multiple branches is that our users frequently deploy or provide multiple versions of a system or product at a time.

Under this model, an application which an organisation may continue to provide bug and security fixes for older versions is unable to easily manage vulnerabilities in multiple supported versions of code using GitLab's integrations. This can lead to users either being forced to use GitLab in unintended ways to facilitate their security scans, or opt to use other scanning tools to avoid this inconvenience.

Goals

- Facilitate multi-branch vulnerability monitoring and historical tracking
- Provide this functionality in a simple, consistent and cohesive way
- Ensure this functionality is well designed and does not pose a risk to GitLab's stability now or in the long term

Proposal

Tracking

In order to facilitate tracking of vulnerabilities across multiple branches, the core of what we are attempting to track can be dissolved into a single sentence:

"What significant detail has occurred regarding this vulnerability at this point in the codebases's history?"

GitLab's vulnerability management system currently works by taking in a list of "Security Findings" which represent the full list of vulnerabilities identified by the scanner in the codebase at that current commit in the repository's history. When we ingest the findings, we reconcile this list of findings against the list of currently known vulnerabilities in the database in order to identify changes in the scan results for that scanner on that repository. Fundamentally the list of things we are identifying when we do this can be boiled down to the list of "states" that we track for vulnerabilities. This list is:

- Detected
- Confirmed
- Dismissed
- Resolved

These states are fundamentally representative of entry and exit points of vulnerability within a codebase. We currently use the `Vulnerabilities::StateTransition` model to track changes between these states for a respective vulnerability. Our current architecture is built with the assumption that a Vulnerability exists only on the default branch of a repository, and so we create a `Vulnerabilities::StateTransition` only when the state of a vulnerability changes on the

default branch.

The vast majority of feature branches do not make any changes to state of a vulnerability within a codebase. This means that we can avoid the most significant risk of database bloat by utilising the branching nature of the git repository to track only the points where these changes occur in a repository's history. We can simply do this by relating the `Vulnerability::StateTransition` to the `Ci::Pipeline` that was the source of the security report that was ingested.

Having the pipeline that was the source of the change in the vulnerability means we can trace a vulnerability's presence in the codebase by retrieving the commit sha associated with the CI pipeline and querying Gitaly using `ListBranchNamesContainingCommitRequest`. This operation will tell us which branches contain the commit sha queried for, allowing us to identify all branches a respective vulnerability exists in.

mermaid

flowchart TD

subgraph Branch: main

B(commit 1)

B --> C(commit 2)

C --> D(commit 3)

D --> E(commit 4)

end

subgraph Branch: Feature 2

E --> F(commit 5)

end

subgraph Vulnerability Resolved

F --> K

K[

Vulnerabilities::StateTransition

toState: :resolved

vulnerabilityid: 1

pipelineid: 5

]

end

B --> G

subgraph New Vulnerability Detected

G[

Vulnerabilities::StateTransition

toState: :detected

vulnerabilityid: 1

pipelineid: 1

]

end

```

subgraph Branch: Feature 1
  C --> I(commit 6)
end

I --> J
subgraph New Vulnerability Detected
  J[
    Vulnerabilities::StateTransition
    tostate: :detected
    vulnerabilityid: 2
    pipelinedid: 6
  ]
end

```

A repository is created and committed to repeatedly. When a vulnerability is resolved in a feature branch we identify this by an appropriate Vulnerability::StateTransition. A vulnerability found in a new branch is defined with a new Vulnerability::StateTransition

```

mermaid
flowchart TD
  subgraph Branch: main
    B(commit 1)
    B --> C(commit 2)
    C --> D(commit 3)
    D --> E(commit 4)
    E --> F(commit 5)
  end

  B --> G
  subgraph New Vulnerability Detected
    G[
      Vulnerabilities::StateTransition
      tostate: :detected
      vulnerabilityid: 1
      pipelinedid: 1
    ]
  end
end

```

```

subgraph Branch: Feature 1
  C --> I(commit 6)
end

I --> J
subgraph New Vulnerability Detected
  J[
    Vulnerabilities::StateTransition
    tostate: :detected

```

```

      vulnerabilityid: 2
      pipelinedid: 6
    ]
  end

  subgraph Vulnerability Resolved
    F --> K
    K[
      Vulnerabilities::StateTransition
      tostate: :resolved
      vulnerabilityid: 1
      pipelinedid: 5
    ]
  end
end

```

The feature branch in which the vulnerability was resolved is merged into the main branch. Because its commit now exists in the main branch, our data does not need to change to still correctly reflect the new state.

All this should be achievable by simply adding a pipelineid column to the vulnerabilitystatetransitions table.

Querying

This information effectively serves as the source of truth regarding the presence and state of a vulnerability in a respective branch. However, in order to present this information to our users and allow them to query and filter it, we need to materialize the information into a state that can be effectively indexed and filtered. Due to a history of performance issues, the current Vulnerability Report works by virtue of a highly denormalized table called vulnerabilityreads which contains all the information related to vulnerabilities in a single row, allowing for effective indexing and filtering.

To minimise the amount of processing we will need to do on demand, we need to store vulnerability reports for different branches in a similar way, effectively "caching" the output for users to filter and query. This is achievable using the existing Vulnerability::Reads paradigm, however we have to acknowledge that we will need substantial storage of denormalised records to make this possible. Each additional branch we wish to track for a project would require us to duplicate their entire body of vulnerabilities, plus or minus the differences specific to the branch.

This should be possible to facilitate by making the following changes to our database structure:

- Partitioning of the Vulnerability Reads table.
 - Tables over a certain size begin to face a wide variety of performance problems. The current size of vulnerabilityreads is already over the threshold which starts facing these problems, so to ensure stable performance going forward we would need to partition.
 - Additionally, per the restrictions at GitLab regarding the adding of columns and indices to tables over a certain size, vulnerabilityreads contravenes both these conditions currently. So

partitioning is not optional in that regard. Though we may have to seek approval to add the additional column.

- Addition of a ref column to the Vulnerability Reads table.
 - By default a ref will simply be a branch name, but if users want to track vulnerabilities by tag, we can allow them to designate tracked tags which can be included in the ref column as well.
- Addition of a partition number to the vulnerabilityreads table.
 - This would allow us to use a sliding list partition strategy for vulnerabilityreads, and can dynamically add new partitions as GitLab scales and users adopt our vulnerability management features to a greater extent.
 - A partition number should be allocated by project/namespace/organisation to minimise data fragmentation. A new partition number should be used when the last partition exceeds 75GB, as this will allow already allocated projects space to grow without exceeding 100GB.
 - Should it be necessary, it should be possible to do partition rebalancing if a particular allocation becomes too heavy.
- Add some kind of table that allows users to designate their desired tracked refs for the project.
 - We can make this include protected branches by default.

Scalability, Storage and Performance

Tracking vulnerabilities across multiple branches will require more everything to facilitate. Materializing vulnerabilities for the vulnerability report so that they can be effectively index and filtered will require as much space again on the vulnerabilityreads table as it took to track the default branch's vulnerabilities. (Plus a bit more for the new ref column and associated changes to indices that will be necessary)

Ingestion of vulnerabilities to update the reports will continue to be an iterative update process associated with the ingestion of security reports from pipelines, so the processing necessary should be nominal in that regard.

However, it is likely that we would not want to keep a materialization of the report at the ready for every branch on every project at all times due to cost reasons. To mitigate this, we can limit the amount of refs that are actively tracked, and then expand this based on our comfort with performance, scalability and cost.

As a mitigation to avoid holding redundant vulnerability report data forever when a user may need it only temporarily, we can probably use the vulnerability history to generate a ref's report on demand, with some UI/UX elements to ask the user to wait while we process and generate the necessary materialised rows. This may mitigate the need to hold too much data when it may only be needed for a short duration.

There are several ways we can tradeoff between branch coverage and the associated storage costs. This tradeoff is not currently in-scope for this design doc. We will chose a justifiable starting point for branch coverage and review our position at a later date.

Because vulnerabilityreads are essentially just a materialized view of information from our source of truth tables, there's no danger in destructing and rebuilding it (as long as we don't disrupt users). As such, should usage grow to such an extent that we need dedicated storage for this denormalized information, this table could be very easily decomposed to a dedicated

database.

Retention

A very important subject in GitLab currently is the correct application of retention policies to data to avoid eternal storage of unused information. We are already in the process of implementing a retention policy for vulnerability information based on the age of the vulnerability.

Because of this new approach to handle vulnerability retention, there should not be any impact on our currently intended 12 month retention policy. Detecting a vulnerability's presence on any branch will refresh the vulnerabilities presence in the codebase and refresh it's retention lifetime.

How to handle history for branch changes

If a branch is deleted, the commits associated with the branch will no longer have a ref tracking them and may be pruned. The associated state transition information may now be redundant, so we should consider pruning it as well.

If a branch is merged and the project is squashing commits with a merge commit, then we may need to consider the merge commit as the new detection/resolution point for the vulnerability and drop the state transition records that were generated for the separate branch. Alternatively, existing Vulnerabilities::StateTransition objects could have their pipelineid updated to match the merge commit.

SBOM dependency tracking for package advisories

In order to track vulnerabilities from package advisories on multiple branches, it becomes necessary to track the dependencies present on all branches as well. Currently there is no architectural consideration for this behaviour in Gitlab, but we may be able to minimally support this by tracking entry/exit points for dependencies in a similar way to how were proposing to use Vulnerability State Transitions to track the same for vulnerabilities. If we know what commits contain a dependency, we can determine what branches have that dependency and then apply the package advisory all applicable branches.

SECTION: engineering/architecture/design-documents/transfer_data/_index.md

{{< engineering/design-document-header >}}

Summary

GitLab already provides users transparency on their Usage Quotas.

We currently display data about:

- used license seats
- used storage

- CI/CD minutes usage

But we don't collect and present transfer data (egress traffic caused by various parts of the application).

Collecting data about number of transferred bytes between clients, customers and services will allow us to discover new efficiencies and reduce the operational risk. We want to better understand the data transfer patterns across the whole application stack.

The goal of this blueprint to describe steps we need to do to achieve the result.

Goals

Explore various solutions to store, process and present transfer data across the whole application stack.

Proposal

There are different types of transferred data.

Type	Description
Repository	Egress data related to Git fetch operations (pull, clone)
Artifacts	Artifacts transfer caused by direct and proxied egress
Pages	Pages egress (depends on Artifacts API)
Packages	Package registry egress
Registry	Container registry egress
Uploads	Object store egress

Each type has different implementations and can be measured separately but collection of metadata / data transfer telemetry and consuming / visualizing it, should be built on top of the same abstraction.

Overview

mermaid
flowchart TB

```
A[Applications] -->|send logs| Pub(Google Pub/Sub)
Pub --> JSONParser
```

```
subgraph DataflowPipeline
  direction TB
```

```
  JSONParser -->|selects only JSON lines| LogProcessor
  LogProcessor -->|insert only data transfer logs| ClickHouse
end
```

ClickHouse -->|query transfer logs| Rails

Applications

Every application produces logs in structured format. Logs related to transfer data requests have metadata fields that include the number of bytes transferred, root namespace ID, project ID, and timestamp of the egress event.

Google Pub/Sub

Application logs are collected and sent to Google Pub/Sub. Pub/Sub allows to subscribe to topics and read incoming logs.

Dataflow pipeline

Dataflow is a Google Cloud unified stream and batch data processing that's serverless, fast, and cost-effective. It's built on the open-source Apache Beam project.

Dataflow pipeline provides a data processing abstraction that can be written in Java, Python or Go.

The Dataflow pipeline is a core of the processing logic. It relies on the streaming implementation of Dataflow. The pipeline subscribes to Pub/Sub topics, reads, processes logs, and inserts them into ClickHouse database.

ClickHouse

ClickHouse is designed to provide a fast access to work with massive data sets. It will allow customers to query aggregated data for dynamic timeframes.

ClickHouse is an abstract store for logs. The Dataflow pipeline will transform different input sources into consistent structure to be stored in ClickHouse. That allows to support various inputs and formats without affecting ClickHouse-stored timeseries.

ClickHouse table schema

sql

```
CREATE TABLE transferdata
(  
  createdat DateTime,  
  bytes UInt64,  
  projectid UInt64,  
  rootnamespaceid UInt64,  
  type String
```


)
ENGINE = MergeTree
PRIMARY KEY (projectid, rootnamespaceid)

- createdat - a timestamp of the event
- bytes - a number of transferred bytes
- projectid - a project ID
- rootnamespaceid - a root namespace ID
- type - a type of egress (git, containerregistry, ...)

Rails

Rails application uses a gem to connect and query ClickHouse.
Customers will be able see their transfer data details in their dashboard.
They can request a transfer data report for their whole namespace or for particular projects.

Implementation proposals

- Repository egress

SECTION: engineering/architecture/design-documents/transfer_data/repository.md

Users generate Repository egress events for every fetch operation in Git. It includes commands like git clone, git fetch and git pull, because all of them request data from Gitaly that needs to be delivered to the end user.

Two main clients of Gitaly traffic are
GitLab Shell(for SSH traffic)
and
Workhorse
(for HTTP traffic).

Both clients send git-upload-pack command to Gitaly and stream back the Git response that contains requested changes.

Current metrics

Service	Number of git-upload-pack events (per day)
-----	-----
Workhorse	~80 million
GitLab Shell	~85 million
Gitaly	~165 million (combined traffic)

Kibana links to see current metrics for each service:

- Workhorse

- GitLab Shell
- Gitaly

Total number of events:

- 165 million per day
- 7.5 million per hour
- 120 thousand per minute

Logs structure

HTTP traffic

Captured in Workhorse logs.

Fields	Description
writtenbytes	number of bytes transferred
uri	namespace and project name
timestamp	timestamp of Egress event

SSH traffic

Captured in GitLab Shell logs.

Fields	Description
writtenbytes	number of bytes transferred
project	full project name
rootnamespace	root namespace
timestamp	timestamp of Egress event

SECTION: engineering/architecture/design-documents/workspaces/_index.md

{{< engineering/design-document-header >}}

Introduction

A common obstacle faced by developers when first contributing to a project is the setup of their local development environment. The time-consuming process of managing interrelated dependencies and fixing version compatibility issues can be demotivating, especially for those who only contribute occasionally or switch between multiple projects frequently. In complicated environments, development teams may also impose restrictions, authentication credentials, build procedures, and standards that add to the fragile nature of the development environment.

At the end of the day, developers want to spend less time managing their development environment and more time contributing high-quality code. And at GitLab, our mission is to ensure everyone can contribute. With Workspaces, our goal is to eliminate the responsibility of

configuring and maintaining a local development environment.

Objectives

See our Workspaces direction page for more information.

Glossary

- 1. GitLab VS Code fork: An internal fork of the VSCode project.
- 1. Web IDE: An IDE built on top of GitLab VS Code fork by adding GitLab context. This IDE runs purely in the browser.
- 1. GitLab VS Code fork for Workspaces: An IDE built on top of GitLab VS Code fork by applying patches making it suitable to use in Workspaces. This IDE runs a server and an optional client which is used to connect remotely.
- 1. GitLab Workflow extension: An extension which adds GitLab features to VS Code.
- 1. GitLab Agent for Kubernetes(agentk): A component for solving any GitLab<->Kubernetes integration tasks.
- 1. GitLab Agent for Workspaces(agentw): A component for solving any GitLab<->Workspace integration tasks.
- 1. GitLab Agent Server(KAS): A server running alongside GitLab Rails application to facilitate connections between different GitLab Agents(agentk, agentw) and GitLab and vice versa.

Overview

Each workspace is run as a group of user provided containers in a Pod in Kubernetes through an integration with GitLab Agent for Kubernetes(agentk).

We inject an IDE(e.g. GitLab VS Code fork for Workspaces) in each Pod. We aim to be IDE agnostic.

The workspace can either be accessed through GitLab Workspaces Proxy deployed in the user's Kubernetes cluster

or through the bi-directional gRPC tunnel established by the GitLab Agent for Workspaces(agentw) injected in the workspace.

Architecture

See Architecture for Kubernetes setup for detailed implementation details.

Decisions

- 1. 001: Configuration format to use
- 1. 002: Provision compute and storage for a workspace
- 1. 003: Authorizing user to create and access a workspace
- 1. 004: Authentication of user traffic to access a workspace
- 1. 005: Explicitly define the user/group ID of the containers at runtime
- 1. 006: Automatic termination of a workspace
- 1. 007: Create workspaces from private projects
- 1. 008: Use SSH to access non-HTTP services running in a workspace
- 1. 009: Allow mapping of GitLab Agent for Kubernetes(agentk) to groups
- 1. 010: Extensions marketplace for GitLab VS Code fork for Workspaces

1. 011: Integrate GitLab Duo features inside a workspace
1. 012: Automatic suspension of a workspace
1. 013: Create workspaces from private container images
1. 014: Default container image and configuration for Workspaces
1. 015: Allow mapping of GitLab Agent for Kubernetes(agentk) to GitLab Cluster
1. 016: Allow users to run sudo commands inside a workspace
1. 017: Allow users to build and run containers inside a workspace
1. 018: Remove GitLab Workspaces Proxy to simplify setup

Helpful Links

- Workspaces direction
- Workspaces | Category Strategy epic
- Workspaces | Market analysis
- Create Remote Development Group YouTube playlist

SECTION: engineering/architecture/design-
documents/workspaces/architecture_kubernetes_setup.md

Overview

Workspaces is delivered as a module(remotedevelopment) in the GitLab agentk for Kubernetes project.

The overall goal of this architecture is to ensure that the actual state of all workspaces running in the Kubernetes clusters is reconciled with the desired state of the workspaces as set by the user.

This is accomplished as follows:

1. The desired state of the workspaces is obtained from user actions in the GitLab UI or API and persisted in the Rails database.
1. There is a reconciliation loop between the agentk and Rails, which:
 - Retrieves the actual state of the workspaces from the Kubernetes clusters through the agentk and sends it to Rails to be persisted.
 - Rails compares the actual state with the desired state and responds with actions to bring the actual state in line with the desired state for all workspaces.

System design

User actions to create/update/delete a workspace

```
plantuml
@startuml
```

```
rectangle "GitLab" {
  rectangle "Nginx/HAProxy" as ReverseProxy
  rectangle "Rails"
  rectangle "Postgres"
```

```
}
```

```
actor "User"  
rectangle "Browser"
```

```
User --> Browser : "1. Browse GitLab"  
Browser --> ReverseProxy : "2: Resolve traffic"  
ReverseProxy --> Rails : "3: Forward\ndecrypted\ntraffic"  
Rails --> Postgres : "4: Persist data for workspace"  
@enduml
```

GitLab Agent for Kubernetes' reconciliation with Rails

```
plantuml  
@startuml  
rectangle "Kubernetes" {  
    rectangle "GitLab Agent for Kubernetes" as AgentK  
    rectangle "Workspace 1" as Workspace1  
    rectangle "Workspace 2" as Workspace2  
    rectangle "Workspace n" as WorkspaceN  
}
```

```
rectangle "GitLab" {  
    rectangle "Rails"  
    rectangle "Postgres"  
    rectangle "GitLab Agent Server(KAS)" as KAS  
}
```

```
AgentK --> KAS : "1: Initiate reconciliation loop"  
KAS --> Rails : "2: Proxy the request\nfrom GitLab Agent\nfor Kubernetes"  
Rails --> Postgres : "3: Fetch workspaces\ndata to be sent to\nGitLab Agent for Kubernetes"  
Rails --> KAS : "4: Respond with\nworkspaces\nKubernetes\nconfig"  
KAS --> AgentK : "5: Proxy the response\nfrom Rails"  
AgentK ..> Workspace1 : "6.1: Applies kubernetes resources\nfor workspace 1"  
AgentK ..> Workspace2 : "6.2: Applies kubernetes resources\nfor workspace 2"  
AgentK ..> WorkspaceN : "6.3: Applies kubernetes resources\nfor workspace N"
```

```
@enduml
```

User accessing the workspace

With GitLab Workspaces Proxy

```
plantuml  
@startuml  
rectangle "Kubernetes" {  
    rectangle "Ingress Controller" as IngressController  
    rectangle "GitLab Workspaces Proxy" as GitLabWorkspacesProxy
```

```
rectangle "Workspace 1" as Workspace1
rectangle "Workspace 2" as Workspace2
rectangle "Workspace n" as WorkspaceN
```

```
note right of IngressController
  Customers can choose
  an ingress controller
  of their choice
end note
```

```
}
```

```
rectangle "GitLab" {
  rectangle "Rails"
}
```

```
actor "User"
rectangle "Browser"
rectangle "Terminal"
rectangle "L7 Load Balancer" as L7LoadBalancer
rectangle "L4 Load Balancer" as L4LoadBalancer
```

```
User -> Browser: "1.1: Browser workspace URL"
User -> Terminal: "1.1: SSH into workspace"
```

```
Browser --> L7LoadBalancer : "2.1: Resolve traffic"
Terminal --> L4LoadBalancer : "2.2: Resolve traffic"
```

```
L7LoadBalancer --> IngressController: "3.1.1: Forward\nencrypted\ntraffic"
IngressController --> GitLabWorkspacesProxy : "3.1.2: Decrypt\ntraffic"
L4LoadBalancer --> GitLabWorkspacesProxy : "3.2: Forward\nTCP traffic"
```

```
GitLabWorkspacesProxy -> Rails : "4: Authenticate and authorize\nthe user accessing the workspace"
```

```
GitLabWorkspacesProxy --> Workspace1 : "5.1. Forward traffic\nfor workspace 1"
GitLabWorkspacesProxy ..> Workspace2 : "5.2. Forward traffic\nfor workspace 2"
GitLabWorkspacesProxy ..> WorkspaceN : "5.3. Forward traffic\nfor workspace N"
```

```
@enduml
```

With GitLab Agent for Workspaces(agentw)

NOTE: The below diagram only reflects the HTTP traffic flow. SSH traffic flow needs investigation and will depend on <https://gitlab.com/groups/gitlab-org/-/epics/13984> .

```
plantuml
@startuml
```

```
rectangle "GitLab" {
```

```
rectangle "Nginx/HAProxy" as ReverseProxy
rectangle "Rails"
rectangle "GitLab\nAgent\nServer\n(KAS)" as KAS
}
```

```
actor "User"
rectangle "Browser"
rectangle "Cloudflare"
rectangle "Workspace Agent\n(agentw)" as AgentW
```

```
note as NoteForCloudflareAntiAbuse
Cloudflare will be used
on .com to provide
anti-abuse features.
Certificate used for
re-encryption is available
with Nginx/HAProxy for
decryption.
end note
```

NoteForCloudflareAntiAbuse .right. Cloudflare

AgentW --> KAS : "1: Start\nngRPC\ntunnel"

User -right-> Browser : "2: Open\nworkspace\nURL"
Browser --> Cloudflare : "3: Resolve traffic"
Cloudflare --> Cloudflare : "4: Decrypt traffic\nand encrypt it back"
Cloudflare --> ReverseProxy : "5: Forward\ntraffic"
ReverseProxy --> ReverseProxy : "6: Decrypt\ntraffic"
ReverseProxy --> KAS : "7: Forward\ntraffic"
KAS -left-> Rails : "8: Authenticate\nand\nauthorize\ntraffic"
Rails --> KAS : "9: Response\nfor\nauthentication\nand\nauthorization"
KAS --> AgentW : "10: Forward\ntraffic\nusing\nreverse\nngRPC\ntunnel"

@enduml

GitLab Agent for Kubernetes topology

- The Kubernetes API is not shown in this diagram, but it is assumed that it is managing the workspaces through the agentk.
- The numbers of components in each Kubernetes cluster are arbitrary.

plantuml
@startuml

title
Workspaces with GitLab agentk for Kubernetes topology
end title

```

rectangle "GitLab" as gitlab {
  rectangle "kas deployment" as kasdeployment {
    collections kas1..kas8
  }
  rectangle rails
  database postgres

  kasdeployment - rails
  rails -left- postgres
}

rectangle "Kubernetes cluster 1" as kubernetes1 {
  rectangle "agentk A workspaces" as agentkaworkspaces {
    collections workspace2..workspace8
    rectangle workspace1
  }

  rectangle "agentk B workspaces" as agentkb..agentkb8workspaces {
    collections workspace10..workspace16
    rectangle workspace9
  }

  rectangle "agentk A deployment" as agentkadeployment {
    rectangle agentka1
  }

  rectangle "agentk B deployment" as agentkbdeployment {
    collections agentkb1..agentkb8
  }

  agentka1 - agentkaworkspaces
  agentkb1..agentkb8 - agentkb..agentkb8workspaces
}

rectangle "Kubernetes cluster 2" as kubernetes2 {
  rectangle "agentk C workspaces" as agentkcworkspaces {
    collections workspace18..workspace24
    rectangle workspace17
  }

  rectangle "agentk C deployment" as agentkcdeployment {
    rectangle agentkc1
  }

  agentkc1 -down- agentkcworkspaces
}

cloud cloud

```



```
cloud - kas1..kas8
cloud - agentka1
cloud - agentkb1..agentkb8
cloud - agentkc1
```

```
'the following hidden line is a hack to get the diagram to render correctly
agentka1 -[hidden]- agentkb1..agentkb8
gitlab -[hidden]d- kubernetes2
```

```
@enduml
```

High-level overview of the communication between Rails and the agentk

```
plantuml
@startuml
!pragma teoz true
```

```
title
  High level overview of the communication between rails and agentk
end title
```

```
box GitLab Beige
participant rails order 20
box kas Bisque
participant "kas" as kas order 40
end box
end box
```

```
box Kubernetes cluster Beige
box agentk Bisque
participant "agentk remotedevelopment\nmodule" as agentkrdmod order 50
end box
participant kubernetes order 60
end box
```

```
loop forever
  agentkrdmod -> kubernetes: Subscribe to Kubernetes for\nworkspace changes associated with the
agentk
  activate agentkrdmod
```

```
  autoactivate on
  agentkrdmod -> kas: POST request with\nupdated workspace information
  note right
    Any updated workspace information from
    Kubernetes is pushed with next reconciliation.
  end note
  kas -> rails: proxy POST request from agentk to rails
```

return Respond with all workspaces to be created/updated/terminated\nalong with an acknowledgement that all information sent by\nagentk have been persisted into the database successfully

return proxy workspace information to agentk
autoactivate off

agentkrdmod -> kubernetes: Apply any received workspace information to kubernetes
deactivate agentkrdmod
end loop

@enduml

Types of messages between Rails and the agentk

The agentk can send different types of messages to Rails to capture different information. Depending on what type of message the agentk sends, Rails will respond accordingly.

Different types of messages are:

- reconcile - Messages sent to Rails to persist the current state of the workspaces. There are two types of updates specified by the updatetype field with the following possible values: full and partial. The payload schema remains the same for both update types.

- full

- Actions performed by the agentk:

- Send the current state of all the workspaces in the Kubernetes cluster managed by the agentk.

- To keep things consistent between the agentk and Rails, the agentk will send this message every time agentk undergoes a full reconciliation cycle that occurs

- when an agentk starts or restarts

- after a leader-election

- periodically, as set using the full reconciliation interval configuration (default: once every hour)

- whenever the agentk configuration is updated

- Actions performed by Rails:

- Update Postgres with the current state and respond with all the workspaces managed by the agentk and their last resource version that Rails has persisted in Postgres.

- Returning the persisted resource version back to the agentk gives it a confirmation that the updates for that workspace have been successfully processed on the Rails end.

- This persisted resource version will also help with sending only the latest workspaces changes from the agentk to Rails for reconcile message with partial update type.

- partial

- Actions performed by the agentk:

- Send the latest workspace changes to Rails that are not yet persisted in Postgres. This persisted resource version will help with sending only the latest workspaces changes from the agentk to Rails.

- Actions performed by Rails:

- Update Postgres with the current state and respond with the workspaces to be created/updated/deleted in the Kubernetes cluster and their last resource version that Rails has persisted in Postgres.

- The workspaces to be created/updated/deleted are roughly calculated by using the filter desired state updated at $\geq$ agentk info reported at.
- Returning the persisted resource version back to the agentk gives it a confirmation that the updates for that workspace have been successfully processed on the Rails end.

Event-driven polling vs full or partial reconciliation

It was initially considered desirable to be able to tell the agentk to not wait for the next reconciliation loop but instead poll immediately. This would grant the following benefits:

1. This would grant the ability to trigger a full reconciliation on demand that would allow on-demand recovery/resetting of module state in the agentk.
1. Apart from making the architecture more event-driven and real-time it would also help to increase the interval between reconciliation polls, thus reducing the load on the infrastructure.

However, as the prospective solutions were evaluated, it was concluded that there are very few/rare cases that would merit this capability, especially given the complexity of the viable options. An eventual reconciliation of state would suffice for most cases and it could be simply achieved through full reconciliation that is carried out periodically (with a longer interval as compared to partial reconciliation).

You can read more in this [issue](#) and [conclusion comment](#)

Workspace states

- CreationRequested - Initial state of a Workspace; Creation requested by user but hasn't yet been acted on
- Starting - In the process of being ready for use
- Running - Ready for use
- Stopping - In the process of scaling down
- Stopped - Persistent storage is still available but workspace has been scaled down
- Failed - Kubernetes resources have been applied by agentk but are not ready due to various reasons (for example, crashing container)
- Error - Kubernetes resources failed to get applied by agentk
- RestartRequested - User has requested a restart of the workspace but the restart has not yet successfully happened
- Terminating - User has requested the termination of the workspace and the action has been initiated but not yet completed.
- Terminated - Persistent storage has been deleted and the workspace has been scaled down
- Unknown - Not able to understand the actual state of the workspace

Possible actualstate values

The actualstate values are determined from the status attribute in the Kubernetes deployment changes, which the agentk listens to and sends to Rails.

The following diagram represents the typical flow of the actualstate values for a Workspace record based on the status values received from the agentk. The status is parsed to derive the actualstate of the

workspace based on different conditions.

However, any of these states can be skipped if there have been any transitional status updates that were not received from the agentk for some reason (a quick transition, a failure to send the event, etc).

plantuml

[] --> CreationRequested

CreationRequested : Initial state before\nworkspace creation\nrequest is sent\nto kubernetes

CreationRequested -right-> Starting : status=Starting

CreationRequested -right-> Error : Could not create\nworkspace

Starting : Workspace config is being\napplied to kubernetes

Starting -right-> Running : status=Running

Starting -down-> Failed : status=Failed\n(container crashing)

Running : Workspace is running

Running -down-> Stopping : status=Stopping

Running -down-> Failed : status=Failed\n(container crashing)

Running -down-> Terminating : status=Terminating

Running -right-> Error : Could not\nstop/terminate\nworkspace

Stopping : Workspace is stopping

Stopping -down-> Stopped : status=Stopped

Stopping -left-> Failed : status=Failed\n(could not\nunmount volume\nand stop workspace)

Stopped : Workspace is Stopped\nby user request

Stopped -left-> Failed : status=Failed\n(could not\nunmount volume\nterminate workspace)

Stopped -right-> Error : Could not\nstart/terminate\nworkspace

Stopped -down-> Terminating : status=Terminating

Terminating : Workspace is terminating

Terminating -down-> Terminated : status=Terminated

Terminated: Workspace has been deleted

Failed: Workspace is not ready due to\nvarious reasons(for example, crashing container)

Failed -up-> Starting : status=Starting\n(container\nnot crashing)

Failed -right-> Stopped : status=Stopped

Failed -down-> Terminating : status=Terminating

Failed -down-> Error : Could not\nstop/terminate\nworkspace

Error: Kubernetes resources failed to get applied

Error -up-> Terminating : status=Terminating

Unknown: Unable to understand the actual state of the workspace

Possible desiredstate values

The desiredstate values are determined from the user's request to Rails and are sent to the agentk by Rails.

desiredstate is a subset of the actualstate with only Running, Stopped, Terminated and RestartRequested values.

The state reconciliation logic in Rails will continually attempt to transition the actualstate to the desiredstate value, unless the workspace is in an unrecoverable state.

There is also an additional supported state of RestartRequested which is only valid for desiredstate.

This value is not a valid value for actualstate. It is required in order for Rails to initiate a restart of a started workspace. It will only persist until a status of Stopped is received

from the agentk, indicating that the restart request was successful and in progress or completed.

At this point, the desiredstate will be automatically changed to Running to trigger the workspace to restart again.

If there is a failure to restart the workspace, and a Stopped status is never received, the desiredstate will remain RestartRequested until a new desiredstate is specified.

plantuml

[] --> Running

Running : Workspace is running

Running -down-> Stopped : status=Stopped

Running -left-> Terminated : status=Terminated

Stopped : Workspace is Stopped\nby user request

Stopped -up-> Running : status=Running

Stopped -down-> Terminated : status=Terminated

Terminated: Workspace has been deleted

RestartRequested : User has requested a workspace restart.\ndesiredstate will automatically change\nto 'Running' if actual state\nof 'Stopped' is received.

RestartRequested -left-> Running : status=Running

SECTION: engineering/architecture/design-

documents/workspaces/decisions/001_configuration_format_to_use.md

Context

To provide a development environment which is reproducible and ephemeral, users need to be able to define their environment in a configuration format. We can choose between adopting an existing development environment configuration format or defining our own.

Decision

We decided to use devfile configuration format because it is highly configurable and it has tools built around it which makes integration easier for us e.g. devfile-library convert devfile to Kubernetes resources.

We will start with devfiles and eventually add support for other configuration formats(e.g. devcontainer) as we receive feedback.

Consequences

Using devfiles means a focus on Kubernetes.

This is acceptable for now since the architecture utilizes it.

Alternatives

For a detailed comparison, see Comparison of devfiles and devcontainers.

We did not consider adding a new configuration format because we are too early in the development stage to know what the requirements would look like and defining a standard at this point would be premature.

SECTION: engineering/architecture/design-
documents/workspaces/decisions/002_provision_compute_and_storage.md

Context

To provide a development environment, we need a compute and storage platform which can be used to dynamically spin up resources as and when required.

Decision

We decided to use Kubernetes because

- It is a CNCF project.
- It is cloud agnostic and provides first class integration with all public clouds to provision compute/storage dynamically.
- It is self-managed first i.e. users can bring in their own infrastructure without us worrying about metering and billing.
- We already have an integration - GitLab Agent for Kubernetes - which allows seamless integration of Kubernetes and GitLab.
- This provides the fastest way to deploy development environments in different private/public clouds without any custom integrations.

We will not provide a managed offering for this yet.

All infrastructure required will be provided by the users and it will be integrated with GitLab by adding an agent module in GitLab Agent for Kubernetes.

Consequences

From past experiences with products launched explicitly on Kubernetes, we have had hurdles with customers since not all customers use Kubernetes or have the expertise to manage a Kubernetes system.

This is acceptable for now since this gives us the fastest way to validate this feature category with customers and iterate on feedback.

Not providing a managed offering might impact adoption/usage since there is some infrastructure expectations from users.

However, providing a managed offering would involve significant more investment to sort out metering and billing.

Alternatives

We briefly considered using CI Runners but could not figure out the logistics of how things would work.

We considered using DevWorkspace Operator since it already supports devfile but did not proceed with it because:

- It is a Kubernetes Custom Resource which means users will have to install a third-party dependency on their Kubernetes clusters.
- DevWorkspace Operator has an additional dependency on cert-manager which is another Kubernetes Custom Resource for cloud native certificate management.
- Adding customizations would be challenging as it would require co-ordinating with upstream project and their releases.
- More details can be found [here](#).

We considered using Eclipse Che but did not proceed with it because

- It relies on DevWorkspace Operator and thus has the same issues as listed above.
- It is opinionated about the ingress controller(traefik) to the Kubernetes cluster.

We initially started with using the DevWorkspace Operator along with GitLab Agent for Kubernetes and ended up removing DevWorkspace Operator dependency for reasons mentioned above as part of removing the dependency on DevWorkspace Operator.

SECTION: engineering/architecture/design-
documents/workspaces/decisions/003_authorizing_user_to_create_and_access_workspace.md

> [!WARNING]

> The decision about which users can create a workspace using which agentk was overridden by 009: Allow mapping of GitLab Agent for Kubernetes(agentk) to groups.

Context

We have decided to use GitLab Agent for Kubernetes in 002: Provision compute and storage for a workspace.

GitLab Agent for Kubernetes(agentk) are tied to a particular project.
We want to provide an easy way to allow projects to use a particular agentk.

Decision

A user can create a workspace from a particular project using all the agentk which are under the group/subgroup as the project. The user should also have Developer access in these groups.

The workspace can only be accessed by the user who has created it.

Consequences

Requiring users to have Developer access in these groups/subgroups can be limiting to many organization.

This is acceptable for now since this gives us the fastest way to validate this feature category with customers and iterate on feedback.

Alternatives

No alternatives were seriously considered at this point because we are at a very early stage and want to get something out for users to try and provide us with feedback.

SECTION: engineering/architecture/design-
documents/workspaces/decisions/004_authentication_of_user_traffic_to_access_workspace.md

Context

We have decided that all infrastructure required will be provided by the users and it will be integrated with GitLab
by adding an agent module in GitLab Agent for Kubernetes in 002: Provision compute and storage for a workspace.

We have decided to only allow the creator of the workspace to access it in
003: Authorizing user to create and access a workspace.

We want a way to enforce these authorization rules in the user's infrastructure.

Decision

We will create a new service which will be deployed in the user's Kubernetes cluster.
This service will be responsible for authenticating and authorizing all user traffic before proxying it to the respective workspace.

All traffic incoming to the Kubernetes cluster on a given domain is forwarded to gitlab-workspaces-proxy, which then decides how to serve that traffic.

mermaid

flowchart TB

```
UserTraffic[User Traffic] --> Ingress
subgraph KubernetesCluster[Kubernetes Cluster]
  Ingress --> GitLabWorkspacesProxy[GitLab Workspaces Proxy]
  GitLabWorkspacesProxy --4.1. Proxy--> Workspace1[Workspace 1]
  GitLabWorkspacesProxy --4.2. Proxy--> Workspace2[Workspace 2]
  GitLabWorkspacesProxy --4.3. Proxy--> Workspace3[Workspace 3]
end
GitLabWorkspacesProxy --1. OAuth 2--> GitLab
GitLab --2. Redirect--> GitLabWorkspacesProxy
GitLabWorkspacesProxy --3. Authorization API--> GitLab
```

More details can be found [here](#).

Consequences

Users will be required to install another component in their Kubernetes cluster along with GitLab Agent for Kubernetes for setting up Workspaces.

Alternatives

We considered using a sidecar proxy approach where the application proxying the traffic will be injected into each workspace and all traffic to the workspace will flow through this proxy. The proxy will only handle the traffic for a single workspace. The Ingress Controller in Kubernetes will forward to the appropriate proxy. The proxy can communicate with the workspace over the loopback interface (localhost) because they share a network namespace as they are in the same Kubernetes Pod.

mermaid

flowchart TB

```
UserTraffic[User Traffic] --> Ingress
subgraph WorkspaceCluster[Workspace Cluster]
  Ingress --> Workspace1Proxy
  Ingress --> Workspace2Proxy
  Ingress --> Workspace3Proxy
  subgraph workspace1[Workspace 1]
    Workspace1Proxy[Workspace Sidecar Proxy] --Proxy--> Workspace1Server[Workspace 1 Server]
  end
  subgraph workspace2[Workspace 2]
    Workspace2Proxy[Workspace Sidecar Proxy] --Proxy--> Workspace2Server[Workspace 2 Server]
  end
  subgraph workspace3[Workspace 3]
    Workspace3Proxy[Workspace Sidecar Proxy] --4.3. Proxy--> Workspace3Server[Workspace 3 Server]
  end
end
end
```

```
Workspace3Proxy --1. OAuth 2--> GitLab
GitLab --2. Redirect--> Workspace3Proxy
```

Workspace3Proxy --3. Authorization API--> GitLab

We did not proceed with this because having a centralized approach made it easier to make changes/upgrades.

We also considered adding auth annotations on the Kubernetes Ingress resource to allow Ingress controllers(for example, ingress-nginx) to delegate authentication and authorization to a separate process. The challenge is that these annotations are not standardized. They are not part of the Ingress specification and may not be supported across different Ingress controllers. We would need to document the process to set up our auth provider for each of the Ingress controllers. This is why we did not proceed with this approach.

SECTION: engineering/architecture/design-
documents/workspaces/decisions/005_explicitly_set_user_group_id_of_containers.md

Context

We have decided to use Kubernetes in 002: Provision compute and storage for a workspace.

We need a way to run securely run multiple untrusted workspaces in Kubernetes.

Decision

With an abundance of caution, we decided to explicitly set the security context with which a Kubernetes Pod will run.
This will prevent privilege escalation.

More details can be found [here](#).

Consequences

Since we clone the project as an init container, the files cloned have a specific user/group ID as the owner.

To allow users to be able to access the files in their workspace with the appropriate file permissions,
we need all the containers to be run as the same user/group ID as the init container cloning the project.

Since we will explicitly set the linux user/group ID of the user which will run the workspace, the container images used to create the workspace should follow the practices around running a container image with arbitrary user ID i.e. at the container build time, the user should not make any assumptions about which user/group ID the container will run as and give file permissions accordingly at build time.

More details can be found [here](#).

Alternatives

No other alternatives were considered.

SECTION: engineering/architecture/design-
documents/workspaces/decisions/006_automatic_termination_of_workspace.md

> [!WARNING]

> The decision was overridden by 012: Automatic suspension of a workspace.

Context

Once workspaces are created, they incur costs for compute and storage until they are terminated.

We allow users to stop their workspaces which will allow them to save the compute costs at least.

However, users do not stop their workspaces when they are done working for the day.

Thus, we need to provide a way for organizations to save money without relying on user expectations.

Decision

In order to avoid unnecessary resource usage and costs, workspaces should be terminated automatically after a given period of time.

More details can be found [here](#).

Consequences

Users can loose their work if they do not save it and the workspace is auto terminated after a given period of time.

Alternatives

No other alternatives were considered.

SECTION: engineering/architecture/design-
documents/workspaces/decisions/007_create_workspace_from_private_projects.md

Context

We want users to be able to create a workspace from private projects.

Decision

We will create Personal Access Token(PAT) associated to the user for each new workspace and

inject it into the workspace.

This token will be used to clone the project as the workspace starts.

This token will be used for all git operations that the user performs while using the workspace.

More details can be found [here](#).

Consequences

The token will be available within the workspace as an environment variable/file.

This can result in token exfiltration if an untrusted container image is used.

This is acceptable for now because users are expected to provision workspaces from projects they trust, like CI jobs.

Alternatives

We considered using ephemeral tokens(JWTs/OAuth/OIDC/etc.)

instead of PAT but each came with its own set of challenges.

This revealed the larger need to have a some standardized authentication and authorization between internal services and GitLab Rails monolith.

SECTION: engineering/architecture/design-
documents/workspaces/decisions/008_use_ssh_to_access_non_http_services.md

Context

We need to provide users a way to access any non-HTTP services running inside the workspace e.g. TCP server the user is running from within the workspace.

Decision

We will use SSH to allow users to access any HTTP/non-HTTP services running inside the workspace.

User can directly SSH into the workspace through the terminal or

use remote/local port-forwarding features of SSH to achieve their goals.

We expect users to have SSH daemon installed and configured in their container images which we will start on workspace startup.

More details can be found [here](#).

Consequences

Not using SSH would mean we need to provide a way to access all the different kinds of traffic

-

TCP, SSH, IMAP, etc. - running in a workspace.

We expect users to have SSH daemon installed and configured in their container images.

Alternatives

No other alternatives were considered.

SECTION: engineering/architecture/design-
documents/workspaces/decisions/009_allow_mapping_of_agentk_to_groups.md

Context

We have decided in 003: Authorizing user to create and access a workspace to only allow a user to create a workspace using any GitLab Agent for Kubernetes where they have at least Developer access in any of group/subgroup of the common ancestor of the project from which the workspace is being created and the project which has the GitLab Agent for Kubernetes registered.

This is limiting in nature because there are many scenarios where users have Developer access only to the project from which they want to create the workspace from and not to any of its parent group.

Furthermore, this approach can be a potential security risk because it allows any user with Developer role within a limited scope to register a GitLab Agent for Kubernetes and have it be potentially used for creating workspaces by users with relevant access to some other projects sharing the hierarchy. Since workspaces contain privileged information such as secrets, more control should be enforced on what GitLab Agent for Kubernetes are available to the user to create a workspaces from.

Decision

We decided to replace the authorization rules introduced in 003: Authorizing user to create and access a workspace to create a workspace with the following rules with some inspiration from the authorization model that is offered by CI/CD runners at GitLab.

- A user can only create workspaces using a GitLab Agent for Kubernetes that is "available", and which has been configured for remotedevelopment.
- A user must have Developer role to both the agent project and the workspace project.
- An agent is considered "available" for use with a workspace if a group owner or administrator has mapped the GitLab Agent for Kubernetes to any parent group of the workspace project. Another way of looking at it is; a mapping between a GitLab Agent for Kubernetes and a group is inherited by its subgroups.
- A GitLab Agent may only be mapped to a parent group.
The group in question may or may not be a direct parent.
For example, if an agent belongs to a project with path root-group/nested-group/agent-project,
then the agent may be mapped to either root-group and/or nested-group.

By default, no GitLab Agent for Kubernetes is mapped to a group.
Additionally, if a project resides within a group, it does NOT imply that

the GitLab Agent for Kubernetes of this project are mapped to the parent group(s).

A GitLab Agent may only be mapped to a group.

In the future, mapping GitLab Agent for Kubernetes to the instance, personal namespaces etc. can/should be explored.

More details can be found [here](#).

Consequences

Since the new rules are incompatible with the existing rules, this feature will be put behind a feature flag and rolled out gradually. Additionally, in order to provide a smooth user experience during feature rollout, a one-time data migration will take place to create mappings between root groups and workspace GitLab Agent for Kubernetes within these groups. After this migration, for any changes desired to the list of GitLab Agent for Kubernetes available during workspace creation, users will be required to explicitly create/delete mappings.

Alternatives

We considered specifying which groups and projects can use the agent to create a new workspace by defining it in the GitLab Agent for Kubernetes configuration.

But this did not address the existing security concerns highlighted above in that it allows a user

with limited privileges to expose infrastructure capabilities for groups/projects that are higher in the hierarchy.

SECTION: engineering/architecture/design-
documents/workspaces/decisions/010_extensions_marketplace_for_vscode.md

Context

We inject GitLab VS Code fork for Workspaces in each workspace.

The VS Code ecosystem relies heavily on extensions to achieve functionalities.

We need to provide a way for users to connect to some VS Code extensions marketplace.

Decision

We decided to connect the GitLab VS Code fork for Workspaces to Open VSX Registry as it is an Eclipse open-source project and alternative to the Visual Studio Marketplace.

More details can be found [here](#) and [here](#).

Consequences

When a workspace starts, GitLab VS Code fork for Workspace will connect to a third-party(Open VSX Registry)

to provide extensions functionality for the IDE. Some users might now want this or might want control over which extensions marketplace they connect to for security or legal

reasons.

This is acceptable for now as we can always add these controls based on user feedback.

Alternatives

We briefly considered building our own custom extensions marketplace backed by a git repository but did not find any reason to prefer it over Open VSX Registry.

SECTION: engineering/architecture/design-
documents/workspaces/decisions/011_integrate_duo_features_inside_workspace.md

Context

GitLab has launched AI capabilities which we should integrate and dogfood.
We need a way to integrate GitLab Duo and related capabilities in a workspace.

Decision

We decided to sideload the GitLab Workflow extension along with the GitLab VS Code fork for Workspaces.
The extension already has integration with GitLab Duo and Duo Chat and other AI features.
The extension will utilize the user's Personal Access Token(PAT) injected inside the workspace for authentication and authorization.

More details can be found [here](#).

Consequences

GitLab Workflow extension would be available in all workspaces along with GitLab Duo and Duo Chat.
This is good because it allows dogfooding of other GitLab projects.

The scope of the PAT injected in the workspace will now include api access.

Alternatives

No other alternatives were considered.

SECTION: engineering/architecture/design-
documents/workspaces/decisions/012_automatic_suspension_of_workspace.md

Context

We decided to automatically terminate workspaces after a given interval of time

in 006: Automatic termination of a workspace.

This results in abrupt termination of a workspace which can result in a loss of work.

We need a way to continue saving costs for users when workspaces are not used actively but not have such actions result in data loss.

Decision

We decided to stop a workspace automatically after a given interval of time instead of terminating.

If the workspace is in a stopped state for a given interval of time, we will then terminate it.

Stopping a workspace results in releasing the compute resources but retaining the storage resources.

This give users an opportunity to restart the workspace if they need it.

At the same time, it will terminate workspace which the user has forgotten about, thus saving resources.

More details can be found [here](#).

Consequences

If there is a long running process inside the workspace, it can be stopped when the workspace stops automatically.

This is acceptable for now because the workspace was being terminated prior to this.

Alternatives

We considered to automatically detect if any user processes are running in a workspace and then stop workspace.

While this might be desired in the long term, this required more investment than what we wanted to invest at this point.

SECTION: engineering/architecture/design-
documents/workspaces/decisions/013_create_workspace_from_private_container.md

Context

Creating workspace from a container image from private container registry requires users to setup

the image pull secrets in Kubernetes which need to be cloned to the namespace of the workspace. This requires additional steps which can be cumbersome.

We need to provide users a way to use container images from private container registries in a secure way.

Decision

We decided to allow users to specify the reference to existing Kubernetes Secrets containing the image pull secret.

When a new workspace is created, these secrets are replicated and auto configured. Any changes to upstream secrets are replicated to all downstream secrets.

There are different container registries each with their own authentication mechanism where the tokens might be ephemeral.

These registries already have tools/documentation to integrate natively with Kubernetes which we will leverage.

More details can be found [here](#).

Consequences

Users will only specify the reference to existing Kubernetes secret and not the secret value itself.

We will leverage the existing tooling the container registries have with Kubernetes.

Alternatives

No other alternative was considered.

SECTION: engineering/architecture/design-
documents/workspaces/decisions/014_default_container_image_and_configuration.md

Context

Users are required to create a devfile in their project to use Workspaces.

This hinders adoption, usage and time-to-first-workspace by making a high barrier to entry.

We need to provide a way for users to create a workspace from a project without any devfile.

Decision

We decided to allow user to create a workspace from a default devfile.

This default devfile will have a container image which will allow users to try out workspaces without having a high barrier to entry.

Consequences

Not providing a default devfile would mean we continue with a high barrier to entry to try out workspaces.

This affects adoption and usage.

Alternatives

We considered providing a list of default devfiles and/or a registry of devfiles for a user to choose from when creating a workspace.

While this might be desired in the long term, this required more investment than what we wanted

to invest at this point.

SECTION: engineering/architecture/design-
documents/workspaces/decisions/015_allow_mapping_of_agentk_to_gitlab_cluster.md

Context

We have decided in 009: Allow mapping of GitLab Agent for Kubernetes(agentk) to groups to allow users to map a GitLab Agent for Kubernetes(agentk) to a group. This requires customers to map the agentk to all groups they want to use Workspaces for. Self-managed customers want an easier way to map an agentk to all the groups in their GitLab cluster.

Decision

We decided to allow users to map a GitLab Agent for Kubernetes(agentk) to a GitLab cluster. This would make it easier for self-managed customers to map one agentk for all projects in their GitLab cluster.

More details can be found [here](#).

Consequences

Not providing an easier way to map an agentk to all projects would create hurdles for customers, thus affecting adoption and usage.

Alternatives

We considered allowing agentk' to be mapped to a namespace under a different top-level group within a GitLab cluster. This would still leave the need for introducing a higher level of mapping(i.e. GitLab cluster).

SECTION: engineering/architecture/design-
documents/workspaces/decisions/016_allow_users_to_run_sudo_commands.md

Context

Development environments often require sudo permissions to install, configure, and use dependencies during runtime. We do not allow users to run any commands with sudo permissions.

We need to allow users to use sudo securely without resulting in privilege escalation.

Decision

Kubernetes does not provide any way to securely run containers inside a container in a Pod. While there are features like User Namespaces in Kubernetes, it is not generally available as of this writing. Thus, customers using any managed Kubernetes offerings - which would presumably include a lot - would not be able to use this as the feature is behind a feature flag in Kubernetes.

We decided to leverage existing tooling built on top of Kubernetes to allow users to run commands with sudo permissions securely inside a container. These existing tooling includes Sysbox and Kata Containers. We also decided to add support for User Namespaces.

More details can be found [here](#).

Consequences

Not providing a way for users to securely run commands with sudo permissions would hamper the value proposition of workspaces which aims to replace local development environments.

Leveraging Sysbox and Kata Containers means we are dependent third-party tools for functionality.

This is acceptable because securely providing elevated permissions inside a container is a tricky and challenging problem which would require tremendous amount of investment. Even then it would be some custom tools on top of Kubernetes which the customers would have to configure. The cost of maintaining such a custom tool would be very high.

Alternatives

We considered using KubeVirt but decided against it since it integrates with Kubernetes using Custom Resources which does not fit our model of integration with Kubernetes.

SECTION: engineering/architecture/design-
documents/workspaces/decisions/017_allow_users_to_build_and_run_containers.md

Context

We decided in 016: Allow users to run sudo commands inside a workspace to allow users to securely use commands with sudo permissions inside a workspace. While this allows for greater flexibility, development environments can require building and running containers to manage and use dependencies during runtime.

We need a way to provide users to build and run containers inside a workspace securely.

Decision

Securely running commands with sudo permissions is a pre-requisite to build and run containers inside a workspace

which was achieved in 016: Allow users to run sudo commands inside a workspace.

We decided to further leverage Sysbox to achieve building and running containers inside a workspace.

M